从零手搭一个最小 Transformer

图文版 · 写给第一次亲手实现 Transformer 的人

由 transformer/README.md 生成 · 2026-10-13 · 446 行正文 · 8 张配图

目录

0. 五分钟跑通（先看到结果，再回头理解）
1. 先想清楚：它到底在算什么
2. 第一步：把文本变成整数（`bpe.py`）
3. 第二步：切分与打包（`corpus.py` / `data.py`）
4. 第三步：模型本体（`model.py`）
5. 第四步：训练循环（`train.py`）
6. 第五步：生成与 KV cache（`generate.py`）
7. 怎么证明「我搭对了」（`test_*.py`）
8. 接下来往哪走

附录 A：两组实测实验（想看「数据 vs 参数」的结论就读这里）
附录 B：换成你自己的语料
附录 C：本仓库的文档与图怎么维护
附录 D：本机环境与排错

> 目标读者：会一点 Python、听说过 Transformer 但没亲手写过的人。读完你能：**从一行文本开始，训出一个会写字的模型**，并且知道每一步为什么这么写。
>
> 全部代码在 `transformer/`，**CPU 就能跑**（8 线程约 19 秒；有 GPU 约 3 秒）。

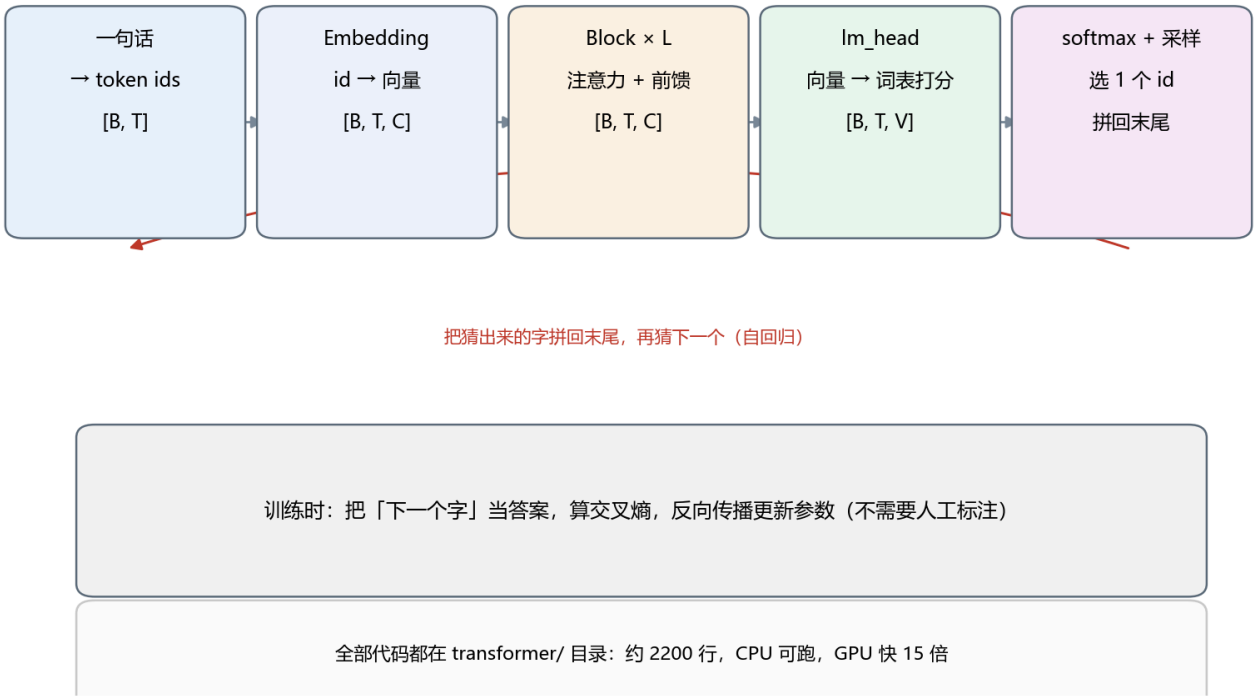


图 1　一张图看懂：Transformer 只做一件事——看着前文猜下一个 token

0. 五分钟跑通（先看到结果，再回头理解）

```
cd /home/yjrrk1989/头脑风暴            # 仓库根目录

# ① 训一个迷你模型：CPU 约 19 秒 / GPU 约 3 秒
PYTHONPATH=. python -m transformer.train --quick          # 有 GPU 加 --device cuda

# ② 让它写 200 个字
PYTHONPATH=. python -m transformer.generate --tag quick -n 200

# ③ 跑 31 条不变量测试（证明搭对了，而不是「看着像」）
PYTHONPATH=. python -m pytest transformer/ -q
```

三条命令**自包含**：不联网、不用下载数据集、不需要 GPU。`--quick` 会在 `runs/quick/` 留下权重和 `history.json`（曲线数据）。

代码一共 10 个文件，职责一眼看清：

文件	干什么	对应哪一节
`bpe.py`	手写字节级 BPE 分词器（零依赖后备）	§2
`corpus.py` / `data.py`	收语料 → 分词 → 打包成 batch	§3
`model.py`	**模型本体**：RMSNorm / RoPE / 注意力 / SwiGLU	§4
`train.py`	训练循环	§5
`generate.py`	采样生成（含 KV cache）	§6
`ladder.py` / `bench.py`	缩放实验 / 性能基准	附录 A
`tutorial.py` / `flowchart.py`	本文所有配图的生成脚本	附录 C
`test_*.py`	31 条不变量测试	§7

1. 先想清楚：它到底在算什么

一句话：**一个「下一个 token 的概率分布」函数。**

```
输入：  注意力机制就是把历史信息按相似度…
输出：                          下一个字 = "加" 0.41 / "权" 0.22 / …
```

就这么一个目标，没有别的花活：

- **训练**：把文本里「下一个字」当答案，算交叉熵、反向传播。**不需要人工标注**——任何文本都是训练数据。
- **推理**：把猜出来的字拼回输入末尾，再猜下一个（这就是「自回归」）。

所以最核心的三行是：

```
logits, loss, _ = model(x, y)   # x 是输入 token，y 是「x 右移一位」
loss.backward()                 # 反向传播
optimizer.step()                # 更新参数
```

2. 第一步：把文本变成整数（`bpe.py`）

模型只吃整数。最朴素的做法是「一个字符一个编号」，但中文和代码会让词表爆炸、序列变长。工业界的答案是 **BPE**：先按字节看（0~255），再反复把「最常一起出现的相邻字节对」合并成一个新符号。

```
"注意力" → UTF-8 字节  e6 b3 a8 e6 b3 a8 e5 8a 9b
        → 合并最高频的一对 (e6,b3) → [X a8 X a8 e5 8a 9b]    X = 新编号 256
        → 继续合并 …              → 常见汉字最后变成 1 个 token
```

自己实现只要 130 行，核心两步：

```
pairs, counts = _pair_stats(tokens)                 # 统计相邻对频次
left, right = pairs[argmax(counts)]                 # 挑最高频的那一对
tokens = _merge_all(tokens, left, right, new_id)    # 合并；新 id = 256 + 第几次合并
```

编码就是**按学到的合并顺序依次施加合并**；解码把每个 id 展开回字节再做 UTF-8 解码，所以**严格可逆。**

三个坑（代码注释里都写了）：

1. **相邻对要「不重叠」地合并**：`a a a` 里的 `(a,a)` 只能合一次，不是两次；
2. 标准库 `re` **不支持** `\p{L}`，GPT-2 的切分正则需要第三方 `regex`（装 `tokenizers` 时自带）。所以 `bpe.py` 在缺 `regex` 时自动降级成更简单的切分；
3. 特殊 token 要占**固定 id**：`<|bos|>` / `<|eos|>`，生成时用 bos 起头、遇 eos 停。

```
PYTHONPATH=. python -m transformer.bpe     # 自测：69 字符 → 36 token，roundtrip OK
```

> 实用经验：数据管线**优先用 HuggingFace `tokenizers` 的 ByteLevel-BPE**（和 Qwen/GPT 同款），手写那份作为**零依赖后备**。两条路径都有 round-trip 测试守着。

3. 第二步：切分与打包（`corpus.py` / `data.py`）

顺序错一步，所有评测数字就都是假的：

```
❌ 先用全部文本训 BPE，再切 train/val   → 分词器见过验证集，val loss 虚低
✅ 先在原始文本上切 train/val，再只用 train 训 BPE
```

然后语言模型有个很漂亮的性质——**标签就是输入自己，错开一位**：

```
x = data[i : i+T]          # 窗口
y = data[i+1 : i+1+T]      # 标签 = 窗口右移一位
```

数据是**流式随机采样**的（把语料当一条长带子，随机剪窗口），所以没有「epoch」概念，也不会过拟合到固定顺序。

```
PYTHONPATH=. python -m transformer.data     # 重建数据（会重训 BPE）
cat data/meta.json                          # token 数 / 词表 / 切分比例
```

4. 第三步：模型本体（`model.py`）

一个 Block 只有两条支路，模型就是把它堆 N 次：

```
class Block(nn.Module):
    def forward(self, x, cos_sin, past_kv=None):
        a, kv = self.attn(self.norm1(x), cos_sin, past_kv)
        x = x + a                          # 残差 1：注意力
        x = x + self.ffn(self.norm2(x))  # 残差 2：前馈
        return x, kv
```

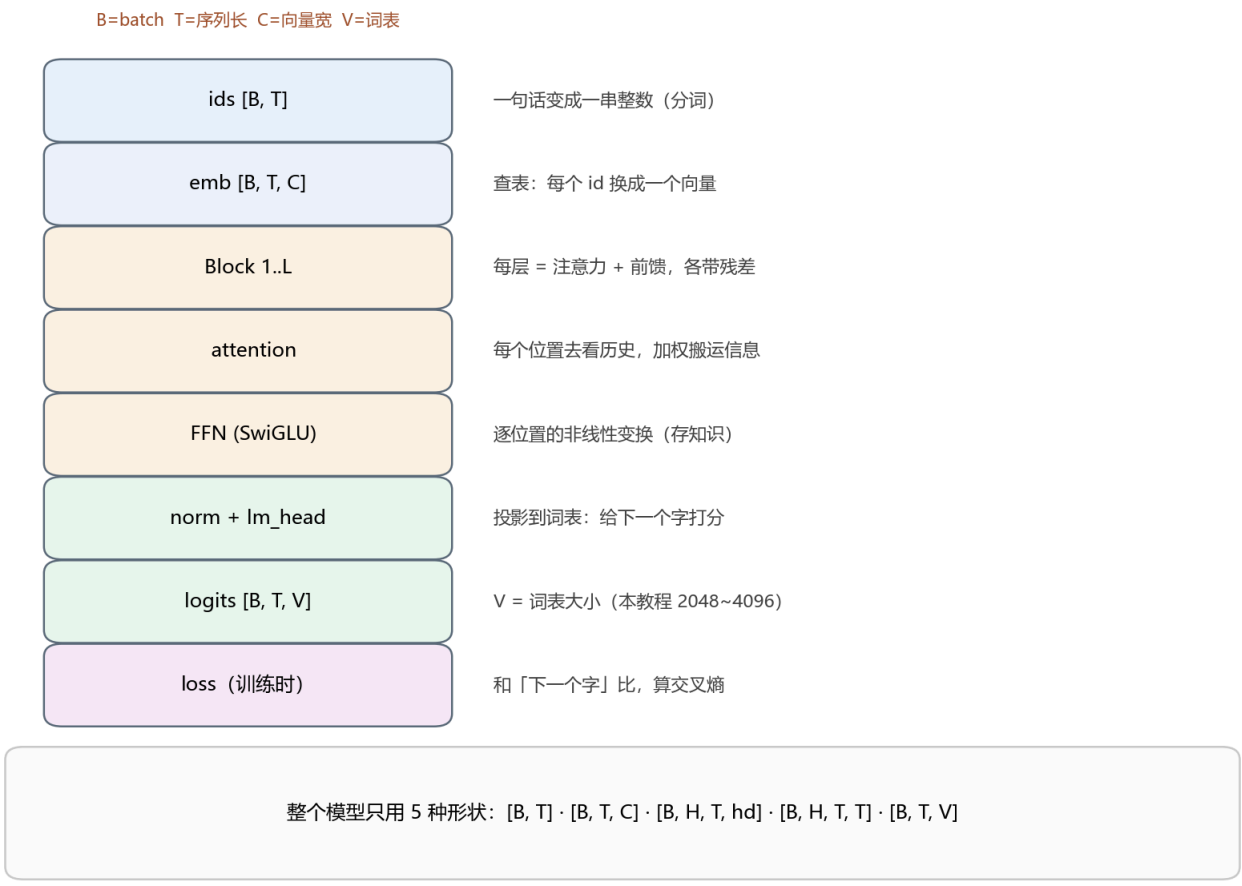


图 2　一次前向的形状变化：从 [B,T] 到 [B,T,V]

整个模型只有 5 种形状，看代码时对照这张表：

符号	含义	本教程
`B`	batch	16
`T`	序列长度	128~512
`C`	向量宽度 `d_model`	128~640
`H` / `hd`	头数 / 每头维度（`hd = C/H`）	4~16 / 32
`V`	词表大小	2048~4096

张量	形状
`ids` / `x` / `y`	`[B, T]`
隐状态	`[B, T, C]`
`q` / `k` / `v`	`[B, H, T, hd]`
注意力权重	`[B, H, T, T]`
`logits`	`[B, T, V]`

4.1 RMSNorm：把向量缩放到单位尺度

```
x / sqrt(mean(x^2) + eps) * weight
```

在**最后一维**求均方；相比 LayerNorm 不减均值、不加偏置——少算一点，效果一样好，是 Llama 之后的默认。`weight` 初值全 1，所以训练开始时它是恒等变换。

4.2 RoPE：用「旋转」注入位置

注意力本身**置换不变**，必须显式加位置信息。RoPE 把 `head_dim` 两两配成若干二维平面，第 `i` 对按角度 `θ_i × 位置` 旋转。妙处是：两个位置的 q·k 内积里，角度**只以 (m−n) 出现**——天然是相对位置，外推性也好。

```
x1, x2 = x[..., 0::2], x[..., 1::2]
return torch.stack([x1*cos - x2*sin, x1*sin + x2*cos], dim=-1).flatten(-2)
```

4.3 注意力：先打分，再按分数取平均

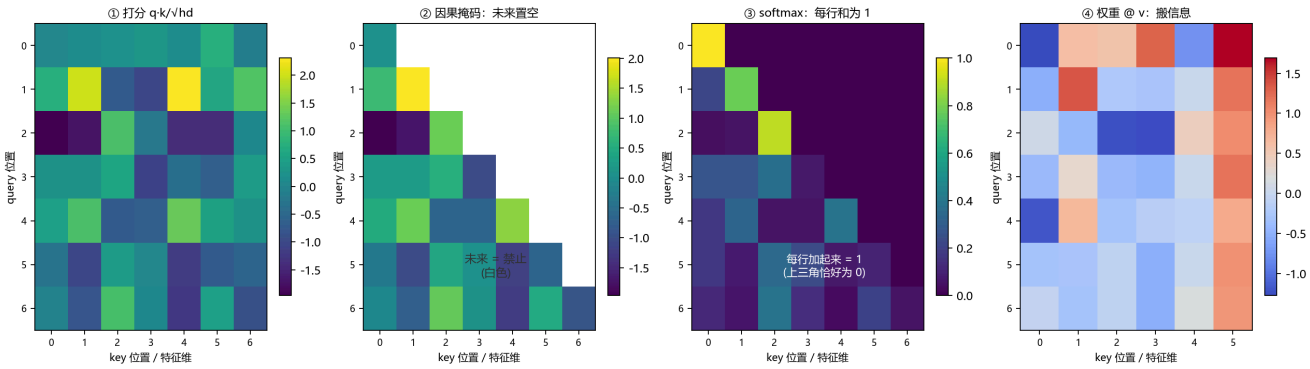


图 3　注意力到底在算什么：先打分，再按分数取平均

```
q = self.q_proj(x)
k, v = self.kv_proj(x).chunk(2, dim=-1)          # GQA 时 kv 头数比 q 少
q, k = apply_rope(q, cos_sin, offset), apply_rope(k, cos_sin, offset)
out = F.scaled_dot_product_attention(q, k, v, attn_mask=allowed)   # 因果掩码在里面
```

四个必须理解的细节：

1. **为什么除以 `√hd`**：`q·k` 是 `hd` 个乘积之和，方差随 `hd` 线性增长；不缩放会把 softmax 推进饱和区，梯度消失。
2. **因果掩码用布尔 `True=可见`** 交给 SDPA：未来位置不可见，softmax 后权重正好是 0。
3. **增量解码（`Tq=1` 且带 cache）根本不需要掩码**——那一个 query 本来就该看全部历史。

4. `v` **不加 RoPE**：位置只该影响「谁关注谁」，不该污染被搬运的内容。

GQA（分组查询注意力）：让每 `n_heads/n_kv_heads` 个 query 头**共享**一组 k/v 头，k/v 的参数量和 **KV cache 都按比例缩小**。`n_kv_heads=n_heads` 就是普通 MHA；测试里用「把 MHA 的权重搬进 GQA 必须逐位相等」钉死了这条等价性。

4.4 SwiGLU 前馈：真正存知识的地方

注意力只做「位置之间的加权平均」（本质是线性的）；单个位置上的非线性变换、事实知识都在 FFN 里，所以 FFN 占参数的大头（每层约 `8d²`）。

```
self.down(F.silu(self.gate(x)) * self.up(x))
```

参数量速算：`≈ 12·L·d² + 2·V·d`。`d=256, L=4, V=2048` 时约 **4.2M**——`train.py` 启动时会打印实际值，**先知道自己在建多大的模型。**

5. 第四步：训练循环（`train.py`）

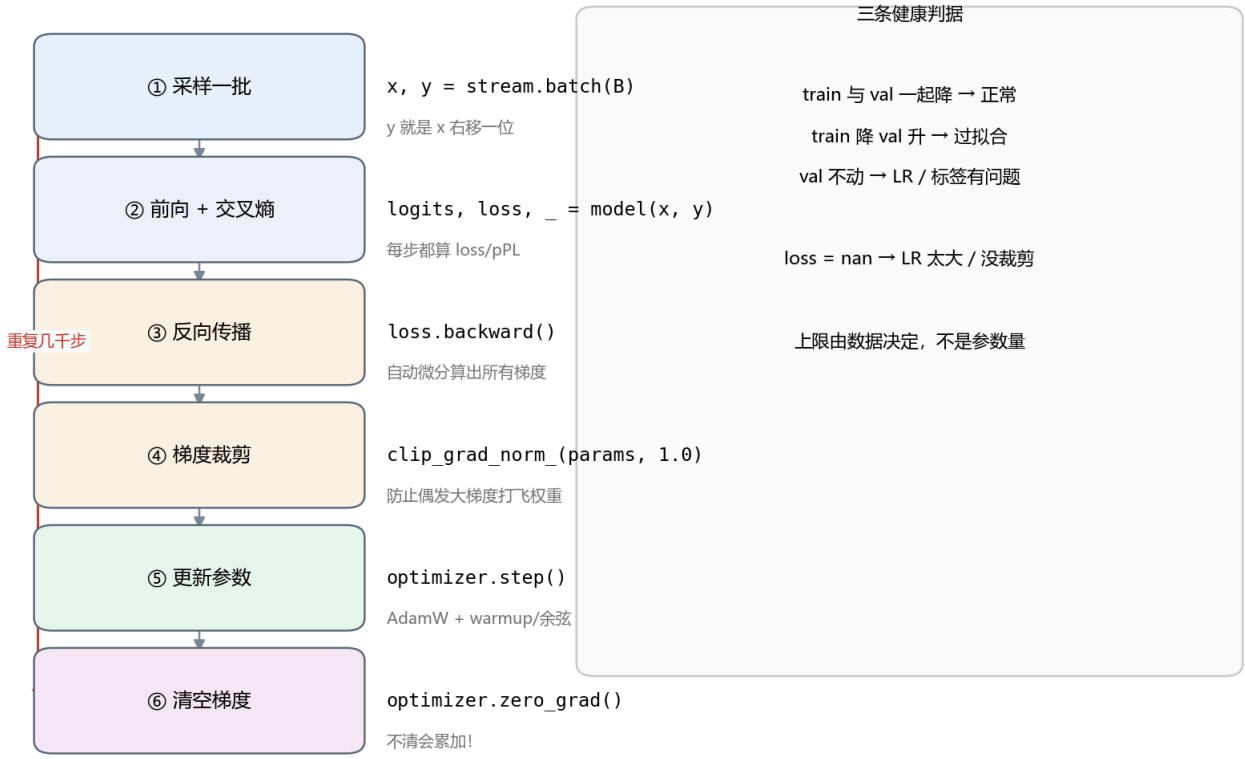


图 5　训练一个 step 的六行代码

```
x, y = stream.batch(B)                                  # ① 采样
logits, loss, _ = model(x, y)                           # ② 前向 + 交叉熵
loss.backward()                                         # ③ 反向：自动微分
gnorm = clip_grad_norm_(model.parameters(), 1.0)        # ④ 梯度裁剪
optimizer.step()                                        # ⑤ AdamW 更新
optimizer.zero_grad(set_to_none=True)                   # ⑥ 清梯度（不清会累加！）
```

学习率用**线性 warmup + 余弦衰减**：刚开始梯度方向不可信，先小步升上来；后期慢慢降下去。

训练曲线怎么读

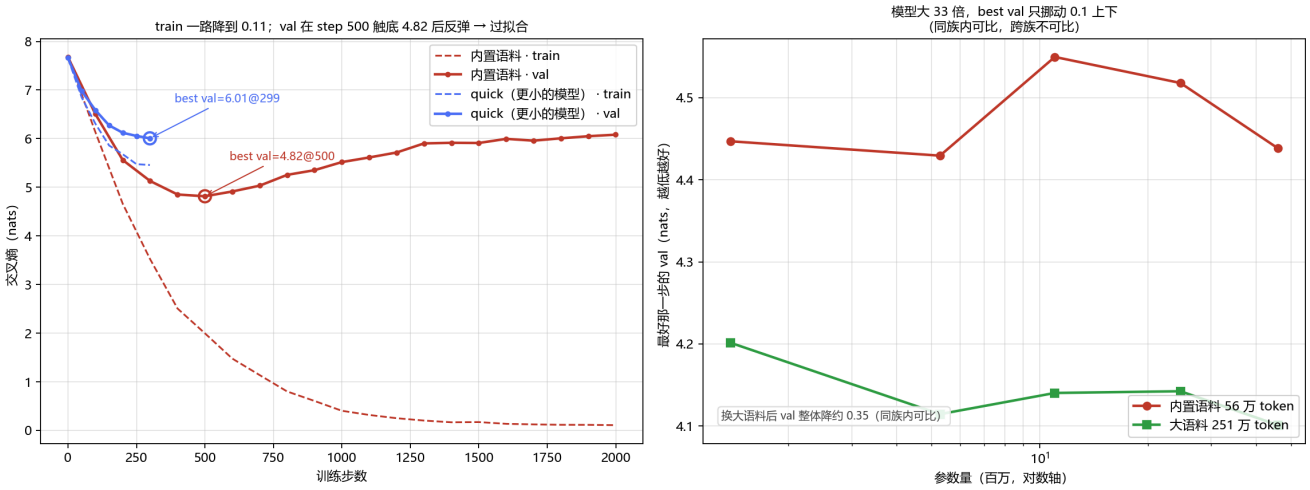


图 6　实测训练曲线：val 先降后升——过拟合长什么样

指标	含义
`loss` / `train` / `val`	交叉熵（nats），越低越好
`ppl = exp(val)`	困惑度：模型在多少个候选之间犹豫
`bits per byte`	**跨分词器可比**的指标：`val/ln2 ÷ 每 token 字节数`
`|g|`	裁剪前的梯度范数

三条健康判据：

- `train` 和 `val` 一起降 → 正常；
- `train` 降、`val` 升 → **过拟合**（图 6 左：train 一直降，val 触底后反弹）；
- `val` 完全不动 → 学习率太小，或标签/数据有问题。

> 图 6 右图提醒一件事：**bpb 的分母依赖 val 文本**，两个数据集之间不能直接比。这是我画图时真犯过的错（第一版把两条线画在同一根轴上），后来改成双轴并明确标注。

6. 第五步：生成与 KV cache（`generate.py`）

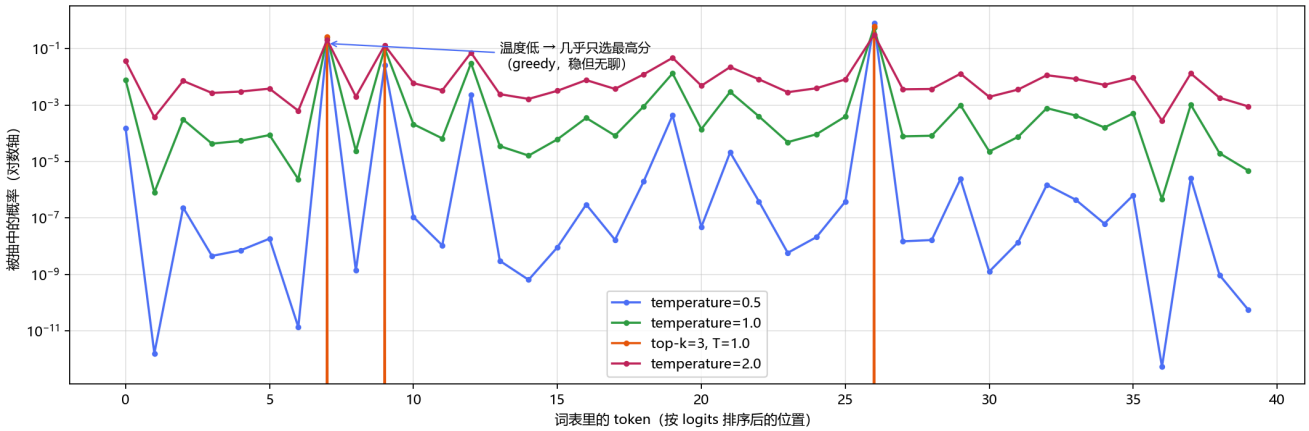


图 4　怎么从 logits 变出一个字：温度与 top-k

```
PYTHONPATH=. python -m transformer.generate --tag quick -p "注意力" -n 200
PYTHONPATH=. python -m transformer.generate --tag quick --time -n 200   # cache 速度对比
```

旋钮	作用	建议
`temperature`	logits 除以 T：小=保守，大=胡言乱语	0.7~1.0
`top_k`	只在概率最高的 k 个里采样，掐掉长尾	40
`use_cache`	每步只前向 1 个 token，复用历史 k/v	默认开

KV cache 为什么快：不缓存时，生成第 n 个 token 要把前 n 个重算一遍（计算量随长度**平方**增长）；缓存后每步只算 1 个新 token。

但它也是最容易写错的地方，三个坑（都有测试守着）：

1. **位置偏移**：增量步的 q/k 必须用「在整段里的绝对位置」，不能从 0 重新数；
2. **掩码平移**：预填充之后 `Tq=1`，不能用 `triu(diagonal=1)`，否则唯一那个 query 被全屏蔽 → NaN；
3. **滑窗**：总长超过 `max_seq_len` 时必须丢最老的 token 并**清空 cache**——第一版漏了这条，生成长文本时崩在 `attn @ v`（尺寸对不上）。

7. 怎么证明「我搭对了」（`test_*.py`）

```
PYTHONPATH=. python -m pytest transformer/ -q      # 31 passed
```

关键性质都写成了**硬断言**，最有用的几条：

测试	验证的性质
`test_attention_is_causal`	**改未来 token，过去的隐状态逐位不变**（+ 反向断言：改未来必须有变化）
`test_rope_is_relative`	两位置同平移分数不变；距离变了分数必须变
`test_kv_cache_matches_full_forward`	增量解码 == 整段重算（1e-5）
`test_generate_past_context_window_with_cache`	超过上下文时滑窗不崩、与 no-cache 一致
`test_gqa_includes_mha_as_special_case`	`n_kv_heads=n_heads` 时与 MHA **逐位等价**
`test_overfit_tiny_sequence`	400 步背下 64 个 token；**换一条序列损失必须 >1**
`test_param_count_scales_as_expected`	参数量公式（防止「以为建了个大模型」）
`test_tutorial_captions_are_in_readme`	8 张教程图的图注仍在正文里（+ 负对照）

「过拟合一条小数据」是搭模型时最有用的调试手段：连 64 个 token 都背不下来，问题一定在代码里（掩码、标签错位、初始化、学习率），不在数据上。

8. 接下来往哪走

按性价比排序：

想干什么	怎么做	代价 / 收益
想让它写得更好	换更大语料：`--data 大文件 --reprepare`（附录 B）	唯一解药，见附录 A 的实测
想更快	`--device cuda`	约 6 倍（实测 19s → 3s；卡越好差距越大）
想省显存	`--n-kv-heads 4`（GQA）	cache 降 1/4；实测生成快 11%、省 35% 显存
想更稳	`--dropout 0.2`	实测 val 3.984 → 3.821（−4.1%），训练慢 51%
想更现代	`--tie-weights`（输出层共享嵌入）、QK-Norm	大词表下省掉整个输出矩阵
想懂训练	打开 `runs/*/history.json` 画曲线	调参靠它，不靠感觉

这套代码故意没做的事：flash-attention（SDPA 会自动挑）、混合精度、分布式、RLHF。把这 5 个形状、2 条残差、1 个掩码吃透之后，那些都是可以按需加装的外挂。

附录 A：两组实测实验（想看「数据 vs 参数」的结论就读这里）

A.1 缩放阶梯：模型变大，指标不动

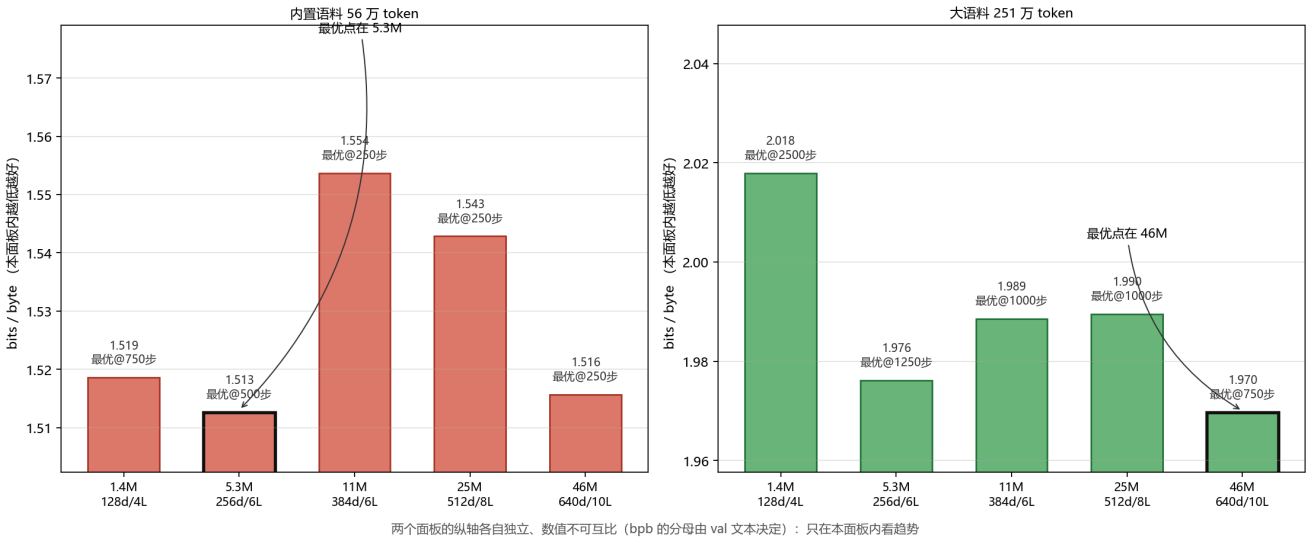


图 7　实测缩放：同一份数据下，增大模型并不带来改善

同一份数据、同样 3000 步、只改模型大小（1.4M → 46M）：bpb 在各自数据集内部几乎不动（**注意两个面板的纵轴不可互比**）：

语料	1.4M	5.3M	11M	25M	46M
内置语料	1.519	**1.513**	1.554	1.543	1.516
大语料（251 万 token）	2.018	1.976	1.989	1.990	**1.970**

结论：在数据受限的区域（这份语料约 0.05 token/参数，Chinchilla 的经验值是 20），加参数的边际收益是负的——模型越大越早背数据，val 触底也越早越浅。**先把数据加够，再谈模型。**

A.2 性能基准：GQA 与 KV cache

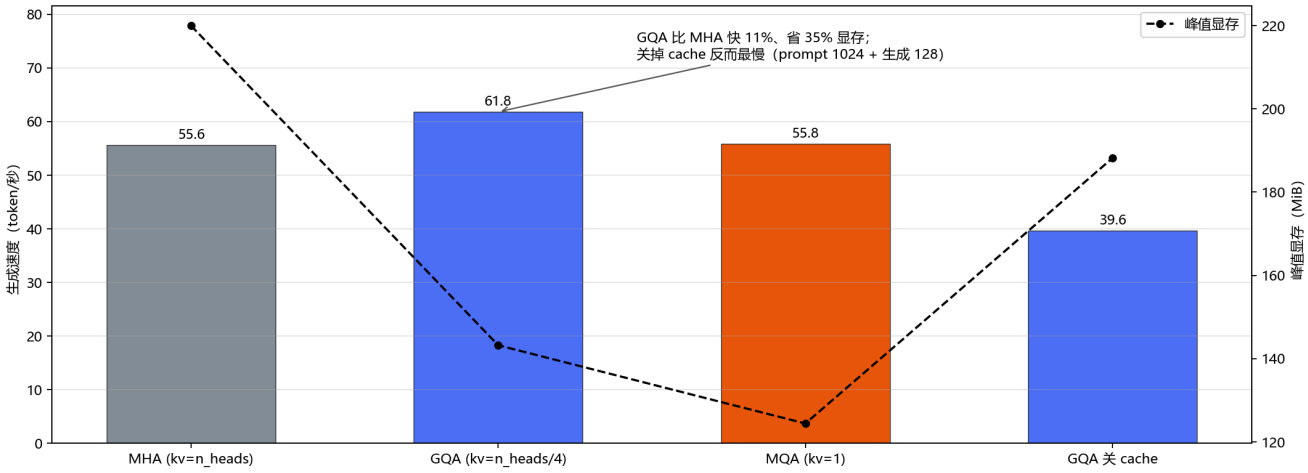


图 8　实测基准：GQA 与 KV cache 在生成时值多少

配置（prompt 512 + 生成 1024）	参数	生成速度	峰值显存
MHA（kv = n_heads）	29.89M	55.6 tok/s	220.0 MiB
GQA（kv = n_heads/4）	26.75M	**61.8 tok/s**	143.2 MiB
MQA（kv = 1）	25.96M	55.8 tok/s	**124.4 MiB**
GQA **关掉 cache**	26.75M	39.6 tok/s	188.2 MiB

KV cache 的收益取决于上下文长度：prompt 512 + 生成 1024 时开 cache 是 1.56×；而 prompt 1024 只生成 128 时，关掉 cache 反而更快（66.2 vs 55.0）——那种情况下「每步只算 1 个 token」的 kernel 启动开销占主导。**别把「cache 一定更快」当教条，按上下文长度实测。**

附录 B：换成你自己的语料

```
# ① 任意文本文件（也支持 .json/.jsonl，会自动抽取正文字段）
PYTHONPATH=. python -m transformer.train --datadir data_mine --data 我的语料.txt --reprepare --
tag mine

# ② 抓一份公开中文语料当练习
PYTHONPATH=. python -m transformer.fetch_corpus --out corpus_big/quan_tang_shi.txt --max-files
25
```

⚠️ **每个语料配一个 `--datadir`**。数据目录里躺着 `tokenizer.json` / `train.bin` / `meta.json`；用 A 语料训的权重配上 B 语料的分词器，生成的会是乱码**而且不报错**。生成时也要指同一个目录：

```
PYTHONPATH=. python -m transformer.generate --datadir data_mine --tag mine -p "开头" -n 200
```

判断「换语料有没有用」只看两个数：**`ckpt.pt` 对应的 val**（不是 `last.pt`）和 **`history.json` 里的 bpb**。val 一路下降不反弹 → 数据还没喂饱，继续加；早早反弹 → 数据不够，调结构没用。

内置语料是怎么组的

`corpus.py` 只扫工作区里**来源明确**的文本（递归 + 白名单），逐层排除产物与自指文件：

来源	内容	字符
根目录 `.py` / `.md`	工作区里的分析脚本与笔记	88.5K
`corpus/rl_notes/*.md`	按文档结构切好的长文归档（11 片）	28.1K
`corpus/src/*.py`	大块源码归档（3 份）	66.0K
`transformer/README.md`	本文（结构最好的中文长文本）	25.1K
合计	26 个文件	**约 208K 字符 / 10.1 万 train token**

排除清单：`transformer/`（代码自身）、`docs/`、`runs/`、`logs/`、`data/`*（产物）*、`rl_corpus.`（自指）。**加新来源就在 `SOURCE_DIRS` / `EXTRA_FILES` 里加一行。**

附录 C：本仓库的文档与图怎么维护

- **图都是脚本生成的**，不手画：`tutorial.py` 出本文这 8 张，`flowchart.py` 出工程流程图。每张图输出三份到 `../docs/`：`.png`（直接看）/ `.svg`（矢量，放大不糊）/ `.mmd` 或 `.txt`（可 diff、可检索）。
- **每张图有锚点**：图注必须逐字出现在本文里，`python -m transformer.tutorial --check` 和测试会断言它仍在。删掉正文的图注会让**测试失败**，而不是留下一张没人引用的图。
- **中文字体降级**：没有中文字体的机器（例如 CI runner）会自动改用英文标签。这条降级路径曾经漏掉脚注和边标签，导致 CI 上渲染出豆腐块——是 CI 抓出来的，见 `../.github/workflows/tests.yml`。
- 重画图请**原地覆盖**；侧栏已打开 `../docs/` 时不要删目录（会让文件树指向已删除的 inode）。

```
PYTHONPATH=. python -m transformer.tutorial     # 重画本文的图
PYTHONPATH=. python -m transformer.flowchart    # 重画工程流程图
PYTHONPATH=. python -m transformer.make_pdf     # 重新生成 README.pdf
```

附录 D：本机环境与排错

现象	原因 / 处理
`ModuleNotFoundError: torch`	用了基础 python；用 `~/miniconda3/envs/grpo/bin/python`
`invalid int value: '200~'`	命令末尾多打了 `~`（`~` 只有写在参数最前面才表示家目录）
`runs/<tag>` 下没有 `.pt`	先训：`python -m transformer.train --quick --tag <tag>`
生成的是乱码	权重与分词器不匹配；`ckpt.pt` 里存了 `tokenizer_meta`，用对应的 `--datadir`
`uint16` 装不下 token id	cl100k 的 id 超过 65535；`data.py` 会按词表自动选 `uint16/uint32`
`re.error: bad escape \p`	标准库 `re` 不支持 `\p{L}`；装 `regex`（`tokenizers` 会自带）
`torch.cuda.is_available() == False`	环境没给 GPU；`train.py` 自动落到 CPU，不用改代码
matplotlib「missing from font」	缺中文字形；生成脚本会打警告并降级为英文标签，不静默输出豆腐块